2026-5-20 Linux常见指令解析：上

- 1. ls 指令
  - 1.1 先认识终端和命令
    - 1.1.1 看懂命令提示符
    - 1.1.2 命令是怎样被执行的
    - 1.1.3 命令的通用格式
  - 1.2 ls 的功能和语法
  - 1.3 常用选项
  - 1.4 读懂 ls -l
    - 1.4.1 长格式的字段
    - 1.4.2 inode 与文件类型
  - 1.5 常用组合示例
    - 1.5.1 查看隐藏文件
    - 1.5.2 按时间查看文件
    - 1.5.3 查看目录本身
  - 1.6 手记补充
- 2. pwd 指令
  - 2.1 功能与语法
  - 2.2 使用场景
- 3. cd 指令
  - 3.1 功能与语法
  - 3.2 常用写法
  - 3.3 理解 Linux 目录树
    - 3.3.1 文件和目录组成一棵树
    - 3.3.2 绝对路径和相对路径
    - 3.3.3 路径不是普通的字符串
  - 3.4 手记补充
- 4. touch 指令
  - 4.1 功能与语法
  - 4.2 常用选项
  - 4.3 用 stat 查看时间
  - 4.4 手记纠错与易错点
- 5. mkdir 指令
  - 5.1 功能与语法
  - 5.2 递归创建目录
  - 5.3 用 tree 检查目录结构

- 6. rmdir 指令与 rm 指令
  - 6.1 rmdir
    - 6.1.1 功能与语法
    - 6.1.2 适用边界
  - 6.2 rm
    - 6.2.1 功能与语法
    - 6.2.2 删除示例
    - 6.2.3 危险边界
  - 6.3 手记补充
- 7. man 指令
  - 7.1 功能与语法
  - 7.2 常用选项
  - 7.3 man 手册的章节
  - 7.4 阅读和退出
- 8. cp 指令
  - 8.1 功能与语法
  - 8.2 常用选项
  - 8.3 典型示例
    - 8.3.1 复制单个文件
    - 8.3.2 用通配符复制多个文件
    - 8.3.3 递归复制目录
  - 8.4 覆盖规则
  - 8.5 手记补充
- 9. mv 指令
  - 9.1 功能与语法
  - 9.2 改名还是移动
    - 9.2.1 改名
    - 9.2.2 移动到目录
    - 9.2.3 同时移动多个对象
  - 9.3 常用选项
  - 9.4 手记补充
- 10. cat 指令
  - 10.1 功能与语法
  - 10.2 常用选项
  - 10.3 用 cat 创建和追加文本
  - 10.4 cat、echo、重定向的关系
  - 10.5 命令速查

1. ls 指令

1.1 先认识终端和命令

1.1.1 看懂命令提示符

登录 Linux 后，最先看到的通常是一行命令提示符：

```
[whb@bite-alicloud ~]$ pwd
/home/whb
```

这一行可以拆成三部分：

- `whb`：当前登录的用户。
- `bite-alicloud`：主机名。
- `~`：当前所处目录。`~` 表示用户的家目录。
- `$`：普通用户的命令提示符；`#` 表示当前是 `root` 用户。
  提示符不是命令的一部分，不需要手动输入。在提示符后输入命令，按回车执行。
  以下几个符号在路径中非常常用：

符号	含义
`.`	当前目录
`..`	上一级目录
`~`	当前用户的家目录
`/`	Linux 根目录
`-`	上一次所在的目录，主要用于 `cd -`

`clear` 可以清空当前终端画面，但不会删除文件，也不会清空前面的历史记录。`Ctrl+L` 通常也能完成同样的操作。

1.1.2 命令是怎样被执行的

Linux 的底层核心叫内核（kernel）。用户通常不直接操作内核，而是通过 Shell 把命令翻译给内核，再把内核的处理结果翻译回终端。

可以把一次执行理解为：

```
用户输入命令 -> Shell 解析 -> 内核执行 -> Shell 输出结果
```

常见的 Shell 是 Bash。注意区分“终端”和“Shell”：终端负责显示和接收输入，Shell 负责解释命令。XShell 只是连接 Linux 的终端工具，真正执行命令的仍然是远端 Linux 上的 Shell。

1.1.3 命令的通用格式

大多数命令都遵循下面的格式：

```
命令 [选项] [参数]
```

例如：

```
ls -a /home
```

- `ls` 是命令。
- `-a` 是选项，用来调整行为。
- `/home` 是参数，用来指定操作对象。

方括号表示“可选”，实际输入时不要输入方括号。

输入命令时还要注意：

1. Linux 命令区分大小写，`ls` 和 `LS` 不是同一个命令。
2. 命令、选项和参数之间用空格分隔。
3. 大多数命令可以直接单独运行，例如 `pwd`。
4. 命令执行后通常回到提示符；如果停在 `:`、`--More--` 或 `Manual page` 界面，先尝试按 `q` 退出。

Linux 中的多数命令本质上都是可执行文件。Shell 会在 `PATH` 环境变量列出的目录中查找命令。当前目录通常不在 `PATH` 中，所以运行当前目录下的程序时，要明确写出 `./`：

```
./my.exe
```

只写 `my.exe` 时，Shell 会去 `PATH` 中查找，而不是自动搜索当前目录，因此可能出现：

```
-bash: my.exe: command not found
```

1.2 `ls` 的功能和语法

`ls` 是 list 的缩写，用来列出目录内容。

```
ls [选项] [目录或文件]
```

- 参数是目录时，列出目录中的文件和子目录。
- 参数是普通文件时，列出该文件及相关信息。
- 不写参数时，默认查看当前目录。

```
ls
ls /home
ls -l /home
```

1.3 常用选项

选项	作用	典型用法
`-a`	显示全部内容，包括以 `.` 开头的隐藏文件	`ls -a`
`-d`	查看目录本身，而不是展开目录内容	`ls -ld dir`
`-i`	显示 inode 编号	`ls -i`
`-k`	以 KB 为单位表示大小，具体输出受系统版本影响	`ls -alk file`
`-l`	以长格式显示详细信息	`ls -l`
`-n`	用数字 UID、GID 代替用户名、组名	`ls -n`
`-F`	在名称后追加符号区分文件类型	`ls -F`
`-r`	反向排序	`ls -r`
`-t`	按修改时间排序，通常新的在前	`ls -t`
`-s`	显示文件占用的存储块数	`ls -s`
`-R`	递归列出子目录内容	`ls -R`
`-1`	一行只显示一个名称	`ls -1`

日常还可以使用 `-h`，让 `-l` 中的大小显示成 `4.0K`、`1.2M` 这样更易读的格式：

```
ls -lh
```

1.4 读懂 `ls -l`

1.4.1 长格式的字段

```
ls -l
```

输出类似：

```
-rw-rw-r-- 1 whb whb 22 Jan 11 15:47 myfile.txt
drwxrwxr-x 2 whb whb 4096 Jan 11 14:22 dir
```

从左到右依次是：

字段	含义
`-rw-rw-r--`	文件类型和三组权限
`1`	硬链接计数
`whb whb`	所有者、所属组
`22`	文件大小
`Jan 11 15:47`	修改时间
`myfile.txt`	文件名

权限字段会结合语法和数字两种方式理解：

```
-  rw-  rw-  r--
|  u    g    o
类型 所有者 同组用户 其他用户
```

`r`、`w`、`x` 分别表示读、写、执行，对应数字 `4`、`2`、`1`。权限的深入内容放到后续权限专题。

1.4.2 inode 与文件类型

`ls -i` 显示的是 inode 编号。inode 用来保存文件的元数据，例如文件类型、权限、大小和时间；文件名则属于目录中的目录项，并不是 inode 本身保存的名字。

```
ls -i
stat myfile.txt
```

`ls -F` 会在名称后追加类型标记：

标记	含义
`/`	目录
`*`	可执行文件
`@`	符号链接
`	`	`
`=`	socket

`-F` 只是帮助识别类型，不会改变文件。还要区分两件事：

- 文件“是什么类型”由文件内容和文件系统元数据决定，不能只看扩展名。
- 文件“能不能执行”由执行权限决定，也不只是看它是不是 `.exe`。

需要进一步判断文件是什么时，可以使用：

```
file myfile.txt
file ./my.exe
```

例如，源码文件即使扩展名为 `.txt`，`file` 仍可能把它识别成 C source。反过来，一个文件即使看起来像可执行程序，只要没有 `x` 权限，也不能直接运行。

1.5 常用组合示例

1.5.1 查看隐藏文件

```
ls -a
ls -al
```

`-a` 会显示 `.` 和 `..`。后者分别是当前目录和上一级目录的目录项。

1.5.2 按时间查看文件

```
ls -lt
ls -ltr
```

`-lt` 让最近修改的文件排在前面；`-ltr` 先按时间排序，再反向排列，可以快速找到较旧或最新的文件。

1.5.3 查看目录本身

```
ls -ld dir
```

如果不加 `-d`，`ls -l dir` 会列出 `dir` 里面的内容；加上 `-d` 才会显示 `dir` 这个目录本身的权限、大小和时间。

1.6 手记补充

> **课堂结论**
>
> 默认的 `ls` 不会显示隐藏文件；`ll` 通常是发行版预先配置的别名，等价于某种形式的 `ls -l`。看到别名时，可以用 `type ll` 或 `which ll` 查看它实际指向什么。

2. pwd 指令

2.1 功能与语法

`pwd` 是 print working directory 的缩写，用来显示当前工作目录的绝对路径。

```
pwd
```

```
pwd
```

输出可能是：

```
/home/whb/test
```

“当前目录”和“家目录”不是同一个概念：

- 当前目录是 Shell 现在正在操作的目录，会随 `cd` 改变。
- 家目录是用户固定拥有的目录，普通用户通常在 `/home/用户名`，`root` 用户通常在 `/root`。

可以用 `whoami` 查看当前用户名：

```
whoami
pwd
```

2.2 使用场景

执行删除、复制、移动或创建文件前，先运行 `pwd` 是避免误操作的好习惯。

```
pwd
ls
```

这两个命令一起使用，可以确认“我现在在哪里”以及“这里有什么”。

3. cd 指令

3.1 功能与语法

`cd` 是 change directory 的缩写，用来切换当前工作目录。

```
cd [目录名]
```

```
cd /home/whb/test
pwd
```

3.2 常用写法

命令	作用
`cd`	返回当前用户的家目录
`cd ~`	返回当前用户的家目录
`cd ~/test`	进入家目录下的 `test`
`cd ..`	返回上一级目录
`cd ../temp`	返回上一级目录，再进入 `temp`
`cd /home`	使用绝对路径进入 `/home`
`cd -`	回到上一次所在目录

示例：

```
pwd
cd ..
pwd

cd /home/whb/test
pwd

cd ~/temp
pwd

cd -
pwd
```

`cd -` 不仅切换目录，通常还会打印切换到的路径。

3.3 理解 Linux 目录树

3.3.1 文件和目录组成一棵树

Linux 中的文件和目录从根目录 `/` 开始，组成一棵树：

- `/` 是根节点。
- `/home`、`/usr`、`/etc` 等是根下的目录。
- 普通文件一定是叶子结点。
- 目录可以是叶子，也可以继续包含文件或子目录。

路径的价值在于从根或当前位置出发，准确描述一个文件或目录的位置。

3.3.2 绝对路径和相对路径

绝对路径从 `/` 开始，不依赖当前所在目录：

```
/home/whb/test/hello.txt
```

无论当前在哪里，这个路径指向的目标都一样，因此适合写在配置文件或服务脚本中。

相对路径从当前目录出发，不以前导 `/` 开始：

```
test/hello.txt
../hello.txt
./hello.txt
```

同一个相对路径在不同当前目录下可能指向不同文件。它更短，交互操作时很常用，但切换目录后必须重新确认它的含义。

> ⚠ **手记纠错**
>
> 绝对路径是“从根目录 `/` 开始”的路径；相对路径是“相对于当前工作目录”的路径。课堂手记中这两句曾写反，复习时以本节为准。

3.3.3 路径不是普通的字符串

`cd dir` 中的 `dir` 是参数，`cd` 会根据文件系统和权限检查这个目标是否存在、能否进入。路径中含有空格时要用引号包起来：

```
cd "my documents"
```

3.4 手记补充

> **两个检查动作**
>
> 切换目录后，用 `pwd` 确认位置，用 `ls` 确认内容。不要只根据终端提示符中的短目录名判断所在位置。

4. touch 指令

4.1 功能与语法

`touch` 有两类常见作用：

1. 文件不存在时，创建一个空的普通文件。
2. 文件已经存在时，更新文件的时间戳，而不清空文件内容。

```
touch [选项] 文件...
```

```
touch newFile.txt
ls -l newFile.txt
```

4.2 常用选项

选项	作用
`-a`	只修改访问时间，即 atime
`-m`	只修改内容修改时间，即 mtime
`-c`	文件不存在时不创建，即 no-create
`-t`	使用指定时间，格式通常为 `[[CC]YY]MMDDhhmm[.ss]`

```
touch -a newFile.txt
touch -m newFile.txt
```

```
touch -c missing.txt
touch -t 202505251230 newFile.txt
```

4.3 用 stat 查看时间

```
stat newFile.txt
```

重点关注三个时间：

时间	含义
Access / atime	最近一次读取或访问内容的时间
Modify / mtime	文件内容最近一次被修改的时间
Change / ctime	inode 元数据最近一次发生变化的时间，不是“创建时间”

`touch -a` 虽然只改变访问时间，但元数据发生了变化，因此 `Change` 时间也可能一起更新。这正是课堂演示中“为什么时间会变化”的原因。

4.4 手记纠错与易错点

> ⚠ **`-c` 不是修改修改时间**
>
> 修改修改时间应使用 `-m`；`-c` 的含义是“文件不存在时不要创建”。两者不要混淆。

`touch` 只能创建文件，不能创建目录。需要目录时应当使用 `mkdir`。

5. mkdir 指令

5.1 功能与语法

`mkdir` 是 make directory 的缩写，用来创建目录。

```
mkdir [选项] 目录名...
```

```
mkdir mydir
ls -ld mydir
```

5.2 递归创建目录

如果父目录不存在，直接创建更深层的目录会失败：

```
mkdir a/b/c
```

使用 `-p` 后，可以自动创建缺失的父目录：

```
mkdir -p path1/path2/path3/path4
tree path1
```

`-p` 还可以在目录已经存在时不报错，因此常用于脚本中的幂等创建：

```
mkdir -p /tmp/my_project/logs
```

5.3 用 tree 检查目录结构

`tree` 会以树状结构展示目录：

```
tree path1
```

它不一定是系统默认安装的命令。在 Ubuntu/Debian 上通常使用：

```
sudo apt update
sudo apt install -y tree
```

在 CentOS/部分 Red Hat 系发行版上通常使用：

```
sudo yum install -y tree
```

包管理器必须与发行版匹配。`yum` 主要是 Red Hat 系的工具，不能直接在 Ubuntu 上照用；Ubuntu 通常使用 `apt`。

6. rmdir 指令与 rm 指令

6.1 rmdir

6.1.1 功能与语法

`rmdir` 用来删除空目录，与 `mkdir` 的“创建目录”相对应。

```
rmdir [-p] 目录名
```

```
mkdir empty
rmdir empty
```

如果目录中还有文件或子目录，`rmdir` 会拒绝删除：

```
rmdir: failed to remove 'path1': Directory not empty
```

`-p` 会尝试删除指定路径以及删除后变成空目录的父目录：

```
rmdir -p path1/path2/path3/path4
```

6.1.2 适用边界

`rmdir` 的安全边界是“只删除空目录”。删除非空目录应使用带 `-r` 的 `rm`，并谨慎确认路径。

6.2 rm

6.2.1 功能与语法

`rm` 用来删除文件或目录。

```
rm [-f] [-i] [-r] [-v] 文件或目录...
```

选项	作用
`-f`	force，强制删除，不提示不存在等问题
`-i`	interactive，每次删除前询问
`-r`	recursive，递归删除目录及其内容
`-v`	verbose，显示删除过程

```
rm file.txt
rm -i file.txt
rm -r dir
rm -rf dir
```

6.2.2 删除示例

删除普通文件前询问：

```
rm -i newFile.txt
```

输出类似：

```
rm: remove regular empty file 'newFile.txt'? y
```

删除整个目录：

```
rm -r dir
```

递归强制删除非空目录：

```
rm -rf path1
```

6.2.3 危险边界

> ⚡ `rm -rf` **没有回收站**
>
> 命令行删除通常不可恢复。执行前先运行 `pwd` 和 `ls`，确认当前目录与目标名称；不要在不确定变量、通配符或路径含义时执行递归删除。

`rm -f` 只删除文件，不加 `-r` 时不能删除目录。`rm -rf` 同时包含递归和强制，风险最高，尤其要避免路径拼写错误和变量为空。

6.3 手记补充

课堂截图分别演示了：删除空目录、删除普通文件前询问、递归删除目录前逐个询问，以及 `rm -rf` 强制递归删除非空目录。命令选项越强，操作前越应该核对当前路径。

7. man 指令

7.1 功能与语法

`man` 用来查看命令、系统调用、库函数、配置文件格式等内容的联机手册。

```
man [选项] 命令
```

```
man ls
man printf
man man
```

7.2 常用选项

用法	作用
`man -k 关键字`	按关键字搜索相关手册
`man 编号 名称`	只查看指定章节
`man -a 名称`	依次查看所有章节中的同名条目

例如：

```
man 2 fork
man 3 printf
man 7 signal
man -k copy
```

7.3 man 手册的章节

不同 Linux 系统可能略有差异，常见划分如下：

1. 普通命令。
2. 系统调用。
3. C 库函数。
4. 特殊文件和设备文件。
5. 配置文件或文件格式。
6. 游戏等杂项。
7. 约定、宏和变量。
8. 系统管理命令。
9. 内核例程等内部接口。

例如，`printf` 既可能是普通命令，也可能是 C 库函数：

- `man 1 printf`：查看命令。
- `man 3 printf`：查看 C 库函数。

`fork` 通常是系统调用，因此可查看 `man 2 fork`。

7.4 阅读和退出

进入 man 页面后常用按键：

按键	作用
`Space`	向下翻一页
`b`	向上翻一页
`/关键字`	向下搜索
`?关键字`	向上搜索
`n`	重复上一次搜索

按键	作用
`N`	反向重复上一次搜索
`q`	退出

不要因为 man 页面停住就关闭终端；按 `q` 返回命令提示符。

8. cp 指令

8.1 功能与语法

`cp` 是 copy 的缩写，用来复制文件或目录。

```
cp [选项] 源文件或目录 目标文件或目录
```

```
cp myfile.txt myfile-backup.txt
```

第二个参数的类型会决定复制行为：

- 目标是普通文件名时，通常复制成这个新文件。
- 目标是已存在目录时，把源文件放进该目录。
- 同时复制多个源时，最后的目标必须是已存在目录。

8.2 常用选项

选项	作用
`-i`	覆盖目标前询问
`-f`	强制覆盖
`-r`	递归复制目录及其内容

```
cp -i myfile.txt myfile-backup.txt
cp -r dir dir-backup
```

如果不确定目标是否已经存在，优先使用 `-i`。不要习惯性使用 `-f`，因为它会减少保护性提示。

8.3 典型示例

8.3.1 复制单个文件

```
cp myfile.txt myfile-backup.txt
cat myfile-backup.txt
```

8.3.2 用通配符复制多个文件

```
mkdir dir
cp *.txt dir/
```

`*` 是 Shell 通配符，会先展开成当前目录下所有以 `.txt` 结尾的文件名，再交给 `cp`。如果没有任何文件匹配，具体行为与 Shell 配置有关。

8.3.3 递归复制目录

```
cp -r test test-backup
tree test-backup
```

普通文件不需要 `-r`；复制目录时必须递归，否则会提示不能复制目录。

8.4 覆盖规则

目标文件已存在时，`cp` 可能直接覆盖。是否默认询问取决于 `cp` 的别名和 Shell 环境。需要稳定地获得提示时，明确写 `-i`。

8.5 手记补充

文件名与内容

用 `echo "hello" > myfile.txt` 可以创建并写入一个带内容的文件。文件是不是文本、图片或程序，要看实际内容；扩展名只是帮助人和工具识别，不是 Linux 判定文件类型的唯一依据。

其中 `>` 由 Shell 处理：目标不存在时创建，存在时覆盖。如果希望追加而不是覆盖，应使用 `>>`。

9. mv 指令

9.1 功能与语法

`mv` 是 move 的缩写，既可以移动文件或目录，也可以改名。

```
mv [选项] 源文件或目录 目标文件或目录
```

```
mv oldname.txt newname.txt
mv file.txt dir/
```

9.2 改名还是移动

9.2.1 改名

目标不存在，或者目标被当作文件名时，源对象会改名为目标名称：

```
touch myfile.txt
mv myfile.txt yourfile.txt
ls
```

9.2.2 移动到目录

目标是已存在目录时，源对象会被移入该目录：

```
mkdir temp
mv myfile.txt temp/
mv temp/ ../
```

9.2.3 同时移动多个对象

```
mv a.txt b.txt dir/
```

最后一个参数必须是已存在的目录。

9.3 常用选项

选项	作用
`-i`	目标已存在时询问是否覆盖
`-f`	强制覆盖，不询问

```
mv -i yourfile.txt myfile.txt
```

`mv` 改名到已有文件名时可能直接覆盖；在重要文件上操作时，加 `-i` 更稳妥。

9.4 手记补充

`mv` 不只是“搬家”，它经常被用来备份和改名。理解第二个参数是“目标文件名”还是“目标目录”，是掌握 `mv` 的关键。

10. cat 指令

10.1 功能与语法

`cat` 来自 concatenate，本义是把多个文件的内容连接起来并输出到标准输出。日常最常见的用法是查看文本文件。

```
cat [选项] [文件...]
```

```
cat myfile.txt
cat file1.txt file2.txt
```

如果文件较大，`cat` 会一次性把内容滚到终端里，不方便定位。此时应使用 `more`、`less`、`head` 或 `tail`，这些命令从下篇继续整理。

10.2 常用选项

选项	作用
`-b`	只对非空行编号
`-n`	对所有输出行编号
`-s`	把连续多个空行压缩成一个空行

```
cat -b myfile.txt
cat -n myfile.txt
cat -s myfile.txt
```

`-b` 和 `-n` 的使用重点不同：

- `-n` 会给每一行编号，包括空行。
- `-b` 只给有内容的行编号。

10.3 用 cat 创建和追加文本

```
cat > note.txt
```

回车后进入键盘输入状态，输入内容后按 `Ctrl+D` 表示输入结束：

```
第一行内容
第二行内容
```

```
cat note.txt
```

追加内容：

```
cat >> note.txt
```

这里真正负责重定向的是 Shell：

- `>` 覆盖写入。
- `>>` 追加写入。

已有文件内容在 `cat > note.txt` 后会丢失，因此对重要文件应先确认，必要时先备份。

10.4 cat、echo、重定向的关系

下面几种写法容易混淆：

```
echo "hello" > note.txt
cat note.txt
cat file1.txt file2.txt > merged.txt
```

第一条命令由 `echo` 产生文本，再由 Shell 写入文件；第二条命令读取文件并显示；第三条把两个文件连接后写入新文件。

> **课堂手记**
>
> 文件的内容决定它大致是什么数据，能不能直接执行还取决于权限。看到 `.exe`、`.sh` 或其他扩展名，都不能跳过 `file` 和权限检查。

10.5 命令速查

任务	命令
查看当前位置	`pwd`
切换目录	`cd 目录`
查看目录内容	`ls -al`
创建空文件	`touch file`
递归创建目录	`mkdir -p a/b/c`
删除空目录	`rmdir dir`
删除目录及内容	`rm -r dir`
查看命令手册	`man 命令`
复制文件	`cp file copy`
递归复制目录	`cp -r dir dir-copy`
改名或移动	`mv source target`
查看短文本	`cat file`

到这里，第 1-10 个基础指令已经串成一条完整的文件操作路径：先用 `pwd` 确认位置，用 `ls` 观察内容，用 `cd` 移动，用 `touch`、`mkdir` 创建，用 `cp`、`mv` 整理，用 `cat` 查看，用 `rm`、`rmdir` 清理，遇到不熟悉的参数再用 `man` 查询。